

DATE:

EXPT. NO.:1

BASIC UNIX COMMANDS

AIM:

To study the basic UNIX commands.

COMMANDS:

1. TASK : To display the system date and time.

COMMAND : date.

SYNTAX : date.

EXPLANATION: This command displays the current system date and time on the screen.

OUTPUT : Tue Jun 19 11:37:17 GMT 2007.

2. TASK : To display the current month.

COMMAND : date.

SYNTAX : date +%m.

EXPLANATION: This command displays the current month on the screen.

OUTPUT : 06.

3. TASK : To display the name of the current month.

COMMAND : date.

SYNTAX : date +%h.

EXPLANATION: This command displays the name of the current month on the screen.

4. TASK : To display the current system date.

COMMAND : date.

SYNTAX : date +%d.

EXPLANATION: This command displays the current system date on the screen.

OUTPUT : 19.

5. TASK : To display the current system date (year).

COMMAND : date.

SYNTAX : date +%y.

EXPLANATION: This command displays the current year on the screen.

OUTPUT : 09.

6. TASK : To display the current system time.

COMMAND : date.

SYNTAX : date +%H.

EXPLANATION: This command displays the current system time (in hours) on the screen.

OUTPUT : 11

7. TASK : To display the current system time.

COMMAND : date.

SYNTAX : date +%M.

EXPLANATION: This command displays the current system time (in minutes) on the screen.

OUTPUT : 43.

8. TASK : To display the current system time.

COMMAND : date.

SYNTAX : date +%S.

EXPLANATION: This command displays the current system time (in seconds) on the screen.

OUTPUT : 15.

9. TASK : To display the calendar of the current month.

COMMAND : calendar.

SYNTAX : cal.

EXPLANATION: This command displays the calendar of the current month on the screen.

OUTPUT : Jun 07

S	M	T	W	T	F	S
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30

10. TASK : To display user-defined message.

COMMAND : echo.

SYNTAX : echo "message".

EXPLANATION: This command displays on the screen the argument of the echo command.

OUTPUT : echo "OS".

□ OS

11. TASK : To display the details of all users.

COMMAND : who.

SYNTAX : who.

EXPLANATION: This command lists the information about all the users who have Logged on to that system.

OUTPUT : student pts/1 2009-02-11 23:56 (83.0.11.200)
Student pts/3 2009-02-11 23:59 (83.0.11.43)

12. TASK : To display the user detail.

COMMAND : who.

SYNTAX : whoami.

EXPLANATION: This command displays information about the current user of the system on the screen.

OUTPUT : root.

13. TASK : To create a directory.

COMMAND : make directory.

SYNTAX : mkdir.

EXPLANATION : This command is used to create a new directory with the specified name.

EXAMPLE : mkdir student.

OUTPUT : The directory “student” is created.

14. TASK : To change directory.

COMMAND : change directory.

SYNTAX : cd directory name.

EXPLANATION : This command is used to switch from one directory to another.

EXAMPLE : cd staff.

OUTPUT : The directory “staff” is switched onto.

15. TASK : To delete a directory.

COMMAND : remove directory.

SYNTAX : rmdir directory name

EXPLANATION : This command is used to delete the specified directory.

EXAMPLE : rmdir student.

OUTPUT : The “student” directory is deleted.

16. TASK : To come out of a sub-directory.

COMMAND : change directory.

SYNTAX : cd ..

EXPLANATION : This command helps in switching to the main directory.

17. TASK : To list all the files and directories.

COMMAND : list.

SYNTAX : ls.

EXPLANATION : This command displays all the files and directories of the system.

18. TASK : To create a file.

COMMAND : cat.

SYNTAX : cat> file name.

EXPLANATION : This command leads to the creation of a new file with the specified file name and contents.

EXAMPLE : cat> wind.

OUTPUT : A null file called “wind” is created.

19. TASK : To view a file.

COMMAND : cat.

SYNTAX : cat file name.

EXPLANATION : This command displays the contents of the specified file.

EXAMPLE : cat wind.

OUTPUT : Contents of the file called “wind” will be displayed on the screen.

20. TASK : To copy a file.

COMMAND : copy.

SYNTAX : cp sourcefile destinationfile.

EXPLANATION : This command produces a copy of the source file and is stored in the specified destination file by overwriting its previous contents.

EXAMPLE : cp sun moon.

OUTPUT : The contents of “sun” file will be copied to the “moon” file.

21. TASK : To move a file.

COMMAND : move.

SYNTAX : mv sourcefile destinationfile.

EXPLANATION : After moving the contents of the source file into destination file, the source file is deleted.

EXAMPLE : mv sun moon.

OUTPUT : After copying contents from the “sun” file to “moon” file, the “sun” file is deleted.

22. TASK : To display / cut a column from a file.

COMMAND : cut.

SYNTAX : cut -c no. file name.

EXPLANATION : This command displays the characters of a particular column in the Specified file.

EXAMPLE : cut -c3 moon.

OUTPUT : Those characters occurring in the 3rd column of the file called “moon” are displayed.

23. TASK : To delete a file.

COMMAND : remove.

SYNTAX : rm file name.

EXPLANATION : This command deletes the specified file from the directory.

EXAMPLE : rm sun.

OUTPUT : The file called “sun” will be deleted.

24. TASK : To retrieve a part of a file.

COMMAND : head.

SYNTAX : head no. of rows file name.

EXPLANATION : This command displays the specified no. of rows form the top of the specified file.

EXAMPLE : head -1 sun.

OUTPUT : The first row of the file called “sun” is displayed.

25. TASK : To retrieve a file.

COMMAND : tail.

SYNTAX : tail no. of rows file name.

EXPLANATION : This command displays the specified no. of rows form the bottom of the specified file.

EXAMPLE : tail -1 moon.

OUTPUT : The last row of the file called “moon” is displayed.

26. TASK : To sort the contents of a file.

COMMAND : sort.

SYNTAX : sort file name.

EXPLANATION : This command helps in sorting the contents of a file in ascending order.

EXAMPLE : sort win.

OUTPUT : The contents of the file “win” are displayed on the screen in a sorted

27. TASK : To display the no. of characters in a file.

COMMAND : word count.

SYNTAX : wc file name.

EXPLANATION : This command displays on the screen the no. of rows, words, and the sum of no. of characters and words.

EXAMPLE : wc ball.

OUTPUT : The no. of rows, words, and no. of characters present in the file “ball” are displayed.

28. TASK : To display the calendar of a year.

COMMAND : cal.

SYNTAX : cal year.

EXPLANATION : This command displays on the screen the calendar of the specified year.

EXAMPLE : cal 2007.

OUTPUT : The calendar of the year 2007 will be displayed.

OUTPUT:

Fedora release 8 (Werewolf)

Kernel 2.6.23.1-42.fc8 on an i686

login: student

Password:

Last login: Thu Feb 12 00:59:23 from 83.0.11.54

[student@localhost ~]\$ mkdir unix

[student@localhost ~]\$ cd unix

[student@localhost unix]\$ cat>file1

welcome to unix

[student@localhost unix]\$ cat>file2

basic commands

[student@localhost unix]\$ cat file1 file2>file3

[student@localhost unix]\$ cat file3

welcome to unix

basic commands

```
[student@localhost unix]$ cp file3 file4
```

```
[student@localhost unix]$ cat file4
```

welcome to unix

basic commands

```
[student@localhost unix]$ mv file4 file5
```

```
[student@localhost unix]$ cat file5
```

welcome to unix

```
[student@localhost unix]$ wc file2
```

```
1 3 16 file2
```

```
[student@localhost unix]$ file file2
```

```
file2: ASCII text
```

```
[student@localhost unix]$ rm file1 file2 file3 file5
```

```
[student@localhost unix]$ cat file3
```

```
cat: file3: No such file or directory
```

```
[student@localhost unix]$ cd ..
```

```
[student@localhost ~]$ rmdir unix
```

```
[student@localhost ~]$ file unix /*
```

```
unix: directory
```

```
/bin: directory
```

```
/boot: directory
```

```
/dev: directory
```

```
/etc: directory
```

```
/home: directory
```

```
/lib: directory
```

```
/lost+found: directory
```

```
/media: directory
```

```
/misc: directory
```

/mnt: directory

/net: directory

/opt: directory

/proc: directory

/root: directory

/sbin: directory

/selinux: directory

/srv: directory

/sys: directory

/tmp: sticky directory

/usr: directory

/var: directory

```
[student@localhost ~]$ cat>file1
```

welcome to unix

```
[student@localhost ~]$ mkdir unix
```

```
[student@localhost ~]$ cd unix
```

```
[student@localhost unix]$ cat>file3
```

its my third file

```
[student@localhost unix]$ cat > file4
```

it is my 4th file

```
[student@localhost unix]$ ls -l
```

total 16

```
-rw-rw-r-- 1 student student 18 2009-02-12 01:12 file3
```

```
-rw-rw-r-- 1 student student 18 2009-02-12 01:13 file4
```

```
[student@localhost unix]$ ls file4 -l
```

```
-rw-rw-r-- 1 student student 18 2009-02-12 01:13 file4
```

```
[student@localhost unix]$ chmod u+x file3
```

```
[student@localhost unix]$ chmod g+x file3
```

```
[student@localhost unix]$ ls file3 -l
```

```
-rwxrwxr-- 1 student student 18 2009-02-12 01:12 file3
```

```
[student@localhost unix]$ pwd
```

```
/home/student/unix
```

```
[student@localhost unix]$ echo UNIX
```

```
UNIX
```

```
[student@localhost unix]$ ls [a-m]*
```

```
file3 file4
```

```
[student@localhost unix]$ date
```

```
Thu Feb 12 01:22:34 IST 2009
```

```
[student@localhost unix]$ date +%m
```

```
02
```

```
[student@localhost unix]$ date +%a
```

```
Thu
```

```
[student@localhost unix]$ cal
```

```
February 2009
```

Su	Mo	Tu	We	Th	Fr	Sa
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28

```
[student@localhost unix]$ cal 2009
```

```
2009
```

25

[student@localhost unix]\$ who

```
student pts/1    2009-02-11 23:56 (83.0.11.200)
student pts/3    2009-02-11 23:59 (83.0.11.43)
student pts/9    2009-02-12 00:02 (83.0.11.45)
student pts/8    2009-02-12 00:04 (83.0.11.49)
student pts/4    2009-02-12 00:06 (83.0.11.46)
student pts/16   2009-02-12 00:17 (83.0.11.39)
student pts/18   2009-02-12 00:19 (83.0.11.52)
student pts/19   2009-02-12 00:20 (83.0.11.50)
student pts/6    2009-02-12 00:22 (83.0.11.56)
student pts/22   2009-02-12 00:24 (83.0.11.37)
student pts/24   2009-02-12 00:25 (83.0.11.41)
student pts/12   2009-02-12 00:26 (83.0.11.48)
student pts/13   2009-02-12 00:28 (83.0.11.7)
student pts/23   2009-02-12 00:38 (83.0.11.8)
student pts/15   2009-02-12 00:41 (83.0.11.156)
student pts/25   2009-02-12 00:42 (83.0.11.9)
student pts/7    2009-02-12 00:48 (83.0.11.38)
student pts/2    2009-02-12 00:49 (83.11.0.35)
student pts/14   2009-02-12 00:50 (83.0.11.36)
student pts/26   2009-02-12 00:51 (83.0.11.59)
student pts/27   2009-02-12 00:51 (83.0.11.47)
student pts/5    2009-02-12 00:58 (83.0.11.44)
student pts/28   2009-02-12 01:01 (83.0.11.200)
student pts/21   2009-02-12 01:08 (83.0.11.42)
student pts/10   2009-02-12 01:09 (83.0.11.54)
student pts/20   2009-02-12 01:11 (83.0.11.55)
student pts/11   2009-02-12 01:24 (83.0.11.53)
```

[student@localhost unix]\$ who am i

```
student pts/28 2009-02-12 01:01 (83.0.11.200)
[student@localhost unix]$ telnet hostname
telnet: hostname: Temporary failure in name resolution
hostname: Host name lookup failure
[student@localhost unix]$ telnet 83.0.11.100
Trying 83.0.11.100...
Connected to 83.0.11.100.
Escape character is '^'.
Fedora release 8 (Werewolf)
Kernel 2.6.23.1-42.fc8 on an i686
login: student
Password:
Last login: Thu Feb 12 01:26:10 from 83.0.11.59
[student@localhost ~]$ which cat
/bin/cat
[student@localhost ~]$ who | wc -l
30
[student@localhost ~]$ who | wc
30 150 1559
[student@localhost ~]$
[student@localhost ~]$ comm file1 file2
unix
Unix
[student@localhost ~]$ diff file1 file2
1c1
< unix
\ No newline at end of file
> Unix
\ No newline at end of file
```

```
[student@localhost ~]$ cat >fil1
```

a

b

c

d

e

f

g

h

i

j

k

l

m

```
[student@localhost ~]$ head fil1
```

a

b

c

d

e

f

g

h

i

j

```
[student@localhost ~]$ tail fil1
```

e

f

g

h

i

j

k

l

m

```
[student@localhost ~]$ cat>sp1
```

jkdfgjk

fghl;j

job

```
[student@localhost ~]$ sort sp1
```

fghl;j

jkdfgjk

job

```
[student@localhost ~]$ spell sp1
```

fghl

jkdfgjk

```
[student@localhost ~]$ uniq sp1
```

jkdfgjk

fghl;j

job

```
[student@localhost ~]$ wc sp1
```

2 3 18 sp1

```
[student@localhost ~]$ cat>st1
```

working in unix

basic unix commands

basic shell commands

c programs in unix

```
[student@localhost ~]$ cat st1 | sort | uniq
```

basic shell commands

basic unix commands

c programs in unix

working in unix

RESULT:

Thus the program was executed successfully and hence output verified.

DATE:

EXPT. NO.:2

PROCESS MANAGEMENT USING SYSTEM CALLS

AIM:

To demonstrate process management in C by creating a child process, replacing its execution image with a system command, and synchronizing the parent process with the child's termination.

ALGORITHM:

1. Start.
2. Initialize a variable pid of type pid_t to store the process ID.
3. Fork: Call fork() to create a child process.
 - If pid < 0: Print an error and exit (fork failed).
 - If pid == 0: You are in the Child Process.
 - Print the child's PID.
 - Use execl() to replace the child's memory space with the ls -l command.
 - If execl() fails, print an error and exit.
 - If pid > 0: You are in the Parent Process.
 - Print the parent's PID and the child's PID being tracked.
 - Call wait(&status) to block execution until the child finishes.
 - Once the child exits, catch the exit status.
 - Print the exit status and resume parent execution.
4. End.

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>


int main() {

    pid_t pid;


    // 1. Create a child process

    pid = fork();


    if (pid < 0) {

        // Error handling if fork fails

        perror("Fork failed");

        exit(EXIT_FAILURE);

    }

    else if (pid == 0) {

        // 2. Child Process: Replace itself with the 'ls' command

        printf("Child Process (PID: %d) is running 'ls'...\n", getpid());


        // execl replaces the current process image with /bin/ls

        execl("/bin/ls", "ls", "-l", NULL);


        // If execl returns, it means it failed
```

```
perror("Exec failed");
exit(EXIT_FAILURE);
}
else {
    // 3. Parent Process: Wait for the child to finish
    int status;

    printf("Parent Process (PID: %d) waiting for Child (PID: %d)...\n",
getpid(), pid);

    wait(&status); // Synchronizing with child termination

    if (WIFEXITED(status)) {
        printf("Child exited with status: %d\n", WEXITSTATUS(status));
    }
    printf("Parent process resuming and ending.\n");
}

return 0;
}
```

OUTPUT:

Parent Process (PID: 1234) waiting for Child (PID: 1235)...

Child Process (PID: 1235) is running 'ls'...

total 16

-rwxr-xr-x 1 user user 16728 Apr 16 01:00 a.out

-rw-r--r-- 1 user user 854 Apr 16 00:58 main.c

Child exited with status: 0

Parent process resuming and ending.

RESULT:

The program successfully demonstrated process management. The `fork()` call split the execution into two paths, the `execl()` call transformed the child into a new program (`ls`), and the `wait()` call ensured the parent did not terminate until the child was finished, preventing "zombie" processes.

DATE:

EXPT. NO.:3a

CPU SCHEDULING – FIRST COME FIRST SERVE

AIM:

To write a C program to implement the FCFS process scheduling mechanisms.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Set the waiting of the first process as '0' and its burst time as its turn around time

Step 5: for each process in the Ready Q calculate

(a) Waiting time for process(n)= waiting time of process (n-1) + Burst time of process(n-1)

(b) Turn around time for Process(n)= waiting time of Process(n)+ Burst time for process(n)

Step 6: Calculate

(a) Average waiting time = Total waiting Time / Number of process

(b) Average Turnaround time = Total Turnaround Time / Number of process

Step 7: Stop the process

PROGRAM:

```
#include<stdio.h>

#include<conio.h>

main()

{
int bt[20], wt[20],tat[20], i, n;

float wtavg, tatavg;

clrscr();

printf("\nEnter the number of processes-- ");

scanf("%d", &n);

for(i=0;i<n;i++){

printf("\nEnterBurstTime for Process%d-- ",i);

scanf("%d",&bt[i]);

}

wt[0] =wtavg = 0;

tat[0] =tatavg =t[0];

for(i=1;i<n;i++){

wt[i] = wt[i-1]+bt[i-1];

tat[i] = tat[i-1] bt[i];

wtavg =wtavg +t[i];

tatavg=tatavg+tat[i];

}

printf("\t PROCESS \tBURST TIME \t WAITING TIME\t TURNAROUND\n");

for(i=0;i<n;i++)

printf("\n\t P%d \t\t %d \t\t %d \t\t %d", i, bt[i],wt[i], tat[i]);
```



```
printf("\nAverage Waiting Time -- %f",wtavg/n);  
printf("\nAverage Turnaround Time --%f", tatavg/n);  
getch();  
}
```

OUTPUT:

INPUT

Enter the number of processes

--

3

Enter Burst Time for Process

0--

24

Enter Burst Time for Process

1--

3

Enter Burst Time for Process

2--

3

OUTPUT

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
P0	24	0	24
P1	3	24	27
P2	3	27	30

Average Waiting Time-- 17.000000

Average Turnaround Time -- 27.000000

RESULT:

Thus the program was executed successfully and hence output verified.

DATE:

EXPT. NO.:3b

CPU SCHEDULING –SHORTEST JOB FIRST

AIM:

To write a C program to implement the SJF process scheduling mechanisms.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Start the Ready Q according the shortest Burst time by sorting according to lowest to highest burst time.

Step 5: Set the waiting time of the first process as '0' and its turnaround time as its burst time.

Step 6: For each process in the ready queue, calculate

(c) Waiting time for process(n)= waiting time of process (n-1) + Burst time of process(n-1)

(d) Turn around time for Process(n)= waiting time of Process(n)+ Burst time for process(n)

Step 6: Calculate

(c) Average waiting time = Total waiting Time / Number of process

(d) Average Turnaround time = Total Turnaround Time / Number of process

Step 7: Stop the process

PROGRAM:

```
#include<stdio.h>

#include<conio.h>

main() {

int p[20], bt[20], wt[20], tat[20], i, k, n, temp;

float wtavg, tatavg;

clrscr();

printf("\nEnter the number of processes -- ");

scanf("%d", &n);

for(i=0;i<n;i++){

p[i]=i;

printf("Enter Burst Time for Process %d -- ", i);

scanf("%d", &bt[i]);

}

for(i=0;i<n;i++)

for(k=i+1;k<n;k++)

if(bt[i]>bt[k]){

temp=bt[i];

bt[i]=bt[k];

bt[k]=temp;

temp=p[i];

p[i]=p[k];

p[k]=temp;

}

wt[0] = wtavg = 0;

tat[0] = tatavg = bt[0];
```

```

for(i=1;i<n;i++)
{
wt[i] = wt[i-1] +bt[i-1];
tat[i] = tat[i-1] +bt[i];
wtavg = wtavg + wt[i];
tatavg = tatavg + tat[i];
}

printf("\n\t PROCESS \tBURST TIME \t WAITING TIME\t TURNAROUND
TIME\n");

for(i=0;i<n;i++)

printf("\n\t P%d \t\t %d \t\t %d \t\t %d", p[i], bt[i], wt[i], tat[i]);

printf("\nAverage Waiting Time -- %f", wtavg/n);

printf("\nAverage Turnaround Time -- %f", tatavg/n);

getch();
}

```

OUTPUT:

INPUT

Enter the number of processes -- 4

Enter Burst Time for Process 0 -- 6

Enter Burst Time for Process 1 -- 8

Enter Burst Time for Process 2 -- 7

Enter Burst Time for Process 3 -- 3

OUTPUT

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
P3	3	0	3
P0	6	3	9
P2	7	9	16
P1	8	16	24

Average Waiting Time -- 7.000000

Average Turnaround Time -- 13.000000

RESULT:

Thus the program was executed successfully and hence output verified.

DATE: _____

EXPT. NO.:3c

CPU SCHEDULING – ROUND ROBIN

AIM:

To write a C program to implement the Round Robin process scheduling mechanisms.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue and time quantum (or) time slice

Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time

Step 4: Calculate the no. of time slices for each process where

$\text{No. of time slice for process}(n) = \text{burst time process}(n) / \text{time slice}$

Step 5: If the burst time is less than the time slice then the no. of time slices =1.

Step 6: Consider the ready queue is a circular Q, calculate

(a) Waiting time for process(n) = waiting time of process(n-1)+ burst time of process(n-1) +

the time difference in getting the CPU from process(n-1)

(b) Turn around time for process(n) = waiting time of process(n) + burst time of process(n)+

the time difference in getting CPU from process(n).

Step 7: Calculate

(c) Average waiting time = Total waiting Time / Number of process

(f) Average Turnaround time = Total Turnaround Time / Number of process

Step 8: Stop the process

PROGRAM:

```
#include<stdio.h>

#include<conio.h>

struct process{
int pn,bt,wt,tat;
}p[10];

void main(){
int tq,i,j=0,n,b[10],ttat=0;
float awt=0,atat=0;
clrscr();
printf("\n enter no. of process:");
scanf("%d",&n);
printf("\n enter input:\np.no\tburs tim\n");
for(i=1;i<=n;i++){
scanf("%d %d",&p[i].pn,&p[i].bt);
b[i]=p[i].bt;
ttat=ttat+p[i].bt;
}
printf("\n enter time quantum:");
scanf("%d",&tq);
while(j<ttat){
for(i=1;i<=n;i++){
if(p[i].bt!=0){
```



```

if(p[i].bt>tq){
p[i].bt=p[i].bt-tq;
j=j+tq;
}else{
j=j+p[i].bt;
p[i].tat=j;
p[i].wt=p[i].tat-b[i];
atat=atat+p[i].tat;
awt=awt+p[i].wt;
p[i].bt=0;
} } }
printf("\np.no\tburst\twait\tturn\n:");
for(i=1;i<=n;i++){
printf("\n%d\t%d\t%d\t%d",p[i].pn,b[i],p[i].wt,p[i].tat);
printf("\n");
}
printf("\n avg wait time is %f",awt/n);
printf("\n avg turn time is %f",atat/n);
getch();
}

```

OUTPUT:

enter no. of process:3

enter input:

p.no	burs tim
------	----------

1	24
---	----

2	3
---	---

3	3
---	---

enter time quantum:4

p.no	burst	wait	turn:
------	-------	------	-------

1	24	6	30
---	----	---	----

2	3	4	7
---	---	---	---

3	3	7	10
---	---	---	----

avg wait time is 5.666667

avg turn time is 15.666667

RESULT:

Thus the program was executed successfully and hence output verified.

DATE:

EXPT. NO.:3d

CPU SCHEDULING – PRIORITY

AIM:

To write a C program to implement the priority process scheduling mechanisms.

ALGORITHM:

1. Start the process
2. Get the number of processes to be inserted
3. Get the corresponding priority of processes
4. Sort the processes according to the priority and allocate the one with highest priority to execute first
5. If two process have same priority then FCFS scheduling algorithm is used
6. Calculate the total and average waiting time and turnaround time
7. Display the values
8. Stop the process

PROGRAM:

```
#include<stdio.h>

main(){
int p[20],bt[20],pri[20], wt[20],tat[20],i, k, n, temp;
float wtavg, tatavg;
clrscr();
```

```

printf("Enter the number of processes --- ");
scanf("%d",&n);
for(i=0;i<n;i++){
p[i] = i;
printf("Enter the Burst Time & Priority of Process %d --- ",i);
scanf("%d %d",&bt[i], &pri[i]);
}
for(i=0;i<n;i++)
for(k=i+1;k<n;k++)
if(pri[i] > pri[k]){
temp=p[i];
p[i]=p[k];
p[k]=temp;
temp=bt[i];
bt[i]=bt[k];
bt[k]=temp;
temp=pri[i];
pri[i]=pri[k];
pri[k]=temp;
}
wtavg = wt[0] = 0;
tatavg = tat[0] = bt[0];
for(i=1;i<n;i++){
wt[i] = wt[i-1] + bt[i-1];
tat[i] = tat[i-1] + bt[i];
wtavg = wtavg + wt[i];

```

```

tatavg = tatavg + tat[i];
}

printf("\nPROCESS\t\tPRIORITY\tBURST TIME\tWAITING
TIME\tTURNAROUND TIME");

for(i=0;i<n;i++)

printf("\n%d \t\t %d \t\t %d \t\t %d \t\t %d ",p[i],pri[i],bt[i],wt[i],tat[i]);

printf("\nAverage Waiting Time is --- %f",wtavg/n);

printf("\nAverage Turnaround Time is --- %f",tatavg/n);

getch();
}

```

OUTPUT:

INPUT

```

Enter the number of processes -- 5
Enter the Burst Time & Priority of Process 0 --- 10 3
Enter the Burst Time & Priority of Process 1 --- 1 1
Enter the Burst Time & Priority of Process 2 --- 2 4
Enter the Burst Time & Priority of Process 3 --- 1 5
Enter the Burst Time & Priority of Process 4 --- 5 2

```

OUTPUT

PROCESS TIME	PRIORITY	BURST TIME	WAITING TIME	TURNAROUND TIME
1	1	1	0	1
4	2	5	1	6
0	3	10	6	16
2	4	2	16	18
3	5	1	18	19

Average Waiting Time is --- 8.200000

Average Turnaround Time is --- 12.000000

RESULT:

Thus the program was executed successfully and hence output verified.

DATE:

EXPT. NO.:4a

IMPLEMENT OF SEMAPHORES

AIM:

To write a program for Implement the Producer – Consumer problem using semaphores (using UNIX system calls).

ALGORITHM:

Step 1: The Semaphore mutex, full & empty are initialized.

Step 2: In the case of producer process

i) Produce an item in to temporary variable.

ii) If there is empty space in the buffer check the mutex value for enters into the critical section.

iii) If the mutex value is 0, allow the producer to add value in the temporary variable to the buffer.

Step 3: In the case of consumer process

i) It should wait if the buffer is empty

ii) If there is any item in the buffer check for mutex value,

iii) if the mutex==0, remove item from buffer

iv) Signal the mutex value and reduce the empty value by 1.

v) Consume the item.

Step 4: Print the result

PROGRAM :

```
#include<stdio.h>

void main() {
    int buffer[10], bufsize, in, out, produce, consume, choice=0;
    in = 0;
    out = 0;
    bufsize = 10;
    while(choice !=3){
        printf("\n1. Produce \t 2. Consume \t 3. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1: if((in+1)%bufsize==out)
                printf("\nBuffer is Full");
            else{
                printf("\nEnter the value: ");
                scanf("%d", &produce);
                buffer[in] = produce;
                in = (in+1)%bufsize;
            }
            Break;
            case 2: if(in == out)
                printf("\nBuffer is Empty");
            else{
                consume = buffer[out];
                printf("\nThe consumed value is %d", consume);
```



```
out = (out+1)%bufsize;  
}  
break;  
}  
}  
}
```

OUTPUT:

1. Produce 2. Consume 3. Exit

Enter your choice: 2

Buffer is Empty

1. Produce 2. Consume 3. Exit

Enter your choice: 1

Enter the value: 100

1. Produce 2. Consume 3. Exit

Enter your choice: 2

The consumed value is 100

1. Produce 2. Consume 3. Exit

Enter your choice: 3

RESULT:

Thus the program for Implement the Producer – Consumer problem using semaphores (using UNIX system calls) was written and successfully executed.

DATE:

EXPT. NO.:4b

IMPLEMENT OF MULTITHREADING

AIM:

To implement the Producer-Consumer problem using Semaphores (mutex, full, and empty) to ensure mutual exclusion and avoid race conditions or buffer overflow/underflow.

ALGORITHM:

Step 1: Initialize the semaphores:

- mutex = 1 (to ensure only one process enters the critical section).
- full = 0 (counts the number of items currently in the buffer).
- empty = N (counts the number of empty slots in the buffer).

Step 2: Producer Process

1. Produce an item into a temporary variable.
2. Check if there is space: wait(empty).
3. Check the mutex to enter the critical section: wait(mutex).
4. Add the item from the temporary variable to the buffer.
5. Release the mutex: signal(mutex).
6. Increment the count of full slots: signal(full).

Step 3: Consumer Process

1. Check if there are items to consume: wait(full).
2. Check the mutex to enter the critical section: wait(mutex).
3. Remove an item from the buffer.
4. Release the mutex: signal(mutex).
5. Increment the count of empty slots: signal(empty).
6. Consume the item.

Step 4: Print the current status of the buffer and the item produced/consumed.

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int mutex = 1, full = 0, empty = 3, x = 0;
```

```
void producer() {  
    --mutex;  // wait(mutex)  
    ++full;   // signal(full)  
    --empty;  // wait(empty)  
    x++;      // Item produced  
    printf("\nProducer produces item %d", x);  
    ++mutex;  // signal(mutex)  
}
```

```
void consumer() {  
    --mutex;  // wait(mutex)  
    --full;   // wait(full)  
    ++empty;  // signal(empty)  
    printf("\nConsumer consumes item %d", x);  
    x--;      // Item consumed  
    ++mutex;  // signal(mutex)  
}
```

```
int main() {  
    int n, i;  
  
    printf("\n1. Press 1 for Producer\n2. Press 2 for Consumer\n3. Press 3 for  
Exit");  
  
    for (i = 1; i > 0; i++) {  
        printf("\n\nEnter your choice: ");  
        scanf("%d", &n);  
  
        switch (n) {  
            case 1:  
                if ((mutex == 1) && (empty != 0)) {  
                    producer();  
                } else {  
                    printf("Buffer is full!");  
                }  
                break;  
            case 2:  
                if ((mutex == 1) && (full != 0)) {  
                    consumer();  
                } else {  
                    printf("Buffer is empty!");  
                }  
            }  
        }  
    }
```

```
        break;
    case 3:
        exit(0);
        break;
    }
}
return 0;
}
```

OUTPUT:

1. Press 1 for Producer
2. Press 2 for Consumer
3. Press 3 for Exit

Enter your choice: 1

Producer produces item 1

Enter your choice: 1

Producer produces item 2

Enter your choice: 2

Consumer consumes item 2

Enter your choice: 2

Consumer consumes item 1

Enter your choice: 2

Buffer is empty!

RESULT:

The implementation ensures that the producer cannot add items to a full buffer and the consumer cannot take items from an empty buffer. The mutex semaphore effectively prevents both from accessing the buffer (critical section) at the same time.

DATE:

EXPT. NO.:5

BANKERS ALGORITHM FOR DEAD LOCK AVOIDANCE

AIM:

Implement the banker's algorithm for dead lock avoidance.

ALGORITHM:

1. Start the program.
2. Get the values of resources and processes.
3. Get the avail value.
4. After allocation find the need value.
5. Check whether its possible to allocate.
6. If it is possible then the system is in safe state.
7. Else system is not in safety state.
8. If the new request comes then check that the system is in safety.
9. or not if we allow the request.
10. stop the program.

PROGRAM:

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main(){
```

```
int req[10], nn, pn, i, j, st[20], s, rn, finish[20], sum[20], count1=0, count=0,  
alloc[20][20], max[20][20], need[20][20], avail[10];
```

```

char sel;

printf("enter no. of process");
scanf("%d",&pn);
printf("enter no. of resources");
scanf("%d",&rn);
for(i=0;i<pn;i++)
finish[i]=0;
printf("enter the allocated resources");
for(i=0;i<pn;i++)
for(j=0;j<rn;j++)
scanf("%d",&alloc[i][j]);
printf("enter the max resources");
for(i=0;i<pn;i++)
for(j=0;j<rn;j++) {
scanf("%d",&max[i][j]);
need[i][j]=max[i][j];
-alloc[i][j];
}
printf("availability of resource is");
for(i=0;i<rn;i++)
scanf("%d",&avail[i]);
for(i=0;i<rn;i++) {
sum[i]=0;
for(j=0;j<pn;j++)
sum[i]=sum[i]+alloc[j][i];
avail[i]=avail[i]-sum[i];}

```



```

printf("Any immediate process");
sel=getch();
if(sel=='y') {
printf("how much they need and which process");
for(i=0;i<rn;i++)
scanf("%d",&req[i]);
scanf("%d",&nn);
count1=0;
for(j=0;j<rn;j++) {
if(req[j]<=need[nn][j])
count1++; }
if(count1==3) {
count1=0;
for(j=0;j<rn;j++) {
if(req[j]<=avail[j])
count1++; }
if(count1==3){
bank++;
printf("process %d is allocated",nn);
printf("\n new avail of this allocated process is");
for(j=0;j<rn;j++) {
alloc[nn][j]=alloc[nn][j]+req[j];
need[nn][j]=need[nn][j]-req[j];
avail[j]=avail[j]-req[j];
printf("\t%d",avail[j]);
} else

```

```

printf("process want to wait"); }
else
printf("error cannot allocate"); }
i=0;s=0;
while(count<pn) {
count1=0;
if(i>=pn)
i=0;
if(finish[i]==0) {
for(j=0;j<rn;j++) {
if(need[i][j]<=avail[j])
count1++; }
if(count1==rn) {
finish[i]=1;
st[s]=i;s++;
count++;
for(j=0;j<rn;j++){
avail[j]=avail[j]+alloc[i][j]; }}
i++;
}
printf("\nsafe state is");
for(i=0;i<pn;i++)
printf(" %d ",st[i]);
getch();
}

```

OUTPUT:

Enter the no of process : 5

Enter the no of resrcs : 3

Enter the allocated resrcs

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter the max resrcs

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Availability of resources : 10 5 7

Any immediate process(y/n) : y

How much they need and which process : 0 4 3 4

Error can't allocate

Safe state is: 1

RESULT:

Thus the program was executed successfully and hence output verified.

DATE:

EXPT. NO.:6

IMPLEMENTATION OF DEADLOCK DETECTION ALGORITHM

AIM:

To implement a Deadlock Detection Algorithm that identifies whether a deadlock exists in a system and determines the processes involved in the deadlock.

ALGORITHM:

- step1. Start the program.
- step2. Get the values of resources and processes.
- step3. Get the avail value.
- step4. After allocation find the need value.
- step5. Check whether its possible to allocate.
- step6. If it is possible then the system is in safe state.
- step7. Else system is not in safety state.
- step8. If the new request comes then check that the system is in safety.
- step9. or not if we allow the request.
- step10. stop the program.

PROGRAM:

```
#include<stdio.h>

#include<conio.h>

void main(){

int cl[10][10],al[10][10],av[10],i,j,k,m,n,ne[10][10],flag=0;

clrscr();

printf("\nEnter the matrix");

scanf("%d %d",&m,&n);

printf("\nEnter the claim matrix:");
```

```

for(i=0;i<m;i++){
for(j=0;j<n;j++){
scanf("%d",&cl[i][j]);
}
}

printf("\nEnter allocated matrix:");

for(i=0;i<m;i++){
for(j=0;j<n;j++)
{
scanf("%d",&al[i][j]);
}
}

printf("\nThe need matrix:\n");

for(i=0;i<m;i++){
for(j=0;j<n;j++){
ne[i][j]=cl[i][j]-al[i][j];
printf("\t%d",ne[i][j]);
}
printf("\n");
}

printf("\nEnter available matrix");

for(i=0;i<n;i++)
scanf("%d",&av[i]);

printf("Claim matrix:\n");

for(i=0;i<m;i++){
for(j=0;j<n;j++){

```

```
printf("\t%d",cl[i][j]);
}
printf("\n");
}
printf("\n Allocated matrix:\n");
for(i=0;i<m;i++){
for(j=0;j<n;j++){
printf("\t%d",al[i][j]);
}
printf("\n");
}
printf("Available matrix:\n");
for(i=0;i<n;i++){
printf("%d\t",av[i]);
}
for(k=0;k<m;k++)
for(i=0;i<m;i++){
for(j=0;j<n;j++){
if(av[j]>=ne[i][j])
flag=1;
else
flag=0;
}
}
if(flag==0)
printf("Unsafe State");
```

```
else
printf("Safe State");
getch();
}
```

OUTPUT:

Input:

Enter the claim matrix:3 2 2 6 1 3 3 1 4 4 2 2

Enter allocated matrix:1 0 0 5 1 1 2 1 1 0 0 2

The need matrix:

2 2 2

1 0 2

1 0 3

4 2 0

Enter available matrix1 1 2

Output:

Claim matrix:

3 2 2

6 1 3

3 1 4

4 2 2

Allocated matrix:

1 0 0

5 1 1

2 1 1

0 0 2

Available matrix:

1 1 2 Safe State

RESULT:

Thus, the Deadlock Detection Algorithm is implemented successfully, and the system state is checked to determine whether a deadlock exists or not.

DATE:

EXPT. NO.:7a

IMPLEMENTATION OF FIRST FIT MEMORY ALLOCATION

AIM:

To implement the First-Fit memory allocation strategy.

ALGORITHM:

Step 1: Start.

Step 2: Input number of memory blocks and their sizes.

Step 3: Input number of processes and their sizes.

Step 4: For each process, scan memory blocks from the beginning.

Step 5: Allocate the first block that is sufficient.

Step 6: Reduce the block size after allocation.

Step 7: Repeat for all processes.

Step 8: Stop.

PROGRAM:

```
#include<stdio.h>
void main()
{
int bsize[10], psize[10], bno, pno, flags[10], allocation[10], i, j;
for(i = 0; i < 10; i++){
flags[i] = 0;
allocation[i] = -1;
}
printf("Enter no. of blocks: ");
scanf("%d", &bno);
printf("\nEnter size of each block: ");
for(i = 0; i < bno; i++)
scanf("%d", &bsize[i]);
printf("\nEnter no. of processes: ");
scanf("%d", &pno);
printf("\nEnter size of each process: ");
for(i = 0; i < pno; i++)
scanf("%d", &psize[i]);
for(i = 0; i < pno; i++) //allocation as per first fit
for(j = 0; j < bno; j++)
if(flags[j] == 0 && bsize[j] >= psize[i])
{
```

```

allocation[j] = i;
flags[j] = 1;
break;
}
//display allocation details
printf("\nBlock no.\tsize\t\tprocess no.\t\tsize");
for(i = 0; i < bno; i++){
printf("\n%d\t\t%d\t\t", i+1, bsize[i]);
if(flags[i] == 1)
printf("%d\t\t\t%d", allocation[i]+1, psize[allocation[i]]);
else
printf("Not allocated");
}
}
}

```

OUTPUT:

INPUT

Enter the number of blocks: 3

Enter the number of files: 2

Enter the size of the blocks:-

Block 1: 5

Block 2: 2

Block 3: 7

Enter the size of the files:-

File 1: 1

File 2: 4

OUTPUT

File No	File Size	Block No	Block Size	Fragment
1	1	1	5	4
2	4	3	7	3

RESULT:

Thus, the First-Fit memory allocation strategy is implemented Successfully.

DATE:

EXPT. NO.: 7b

IMPLEMENTATION OF BEST FIT MEMORY ALLOCATION

AIM:

To implement the Best-Fit memory allocation strategy.

ALGORITHM:

Step 1: Start.

Step 2: Input memory blocks and process sizes.

Step 3: For each process, find the smallest block that is sufficient.

Step 4: Allocate that block.

Step 5: Reduce the block size after allocation.

Step 6: Repeat for all processes.

Step 7: Stop.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
#define max 25
void main()
{
int frag[max],b[max],f[max],i,j,nb,nf,temp,lowest=10000; static int
bf[max],ff[max];
clrscr();
printf("\nEnter the number of blocks:"); scanf("%d",&nb);
printf("Enter the number of files:"); scanf("%d",&nf);
printf("\nEnter the size of the blocks:-\n"); for(i=1;i<=nb;i++)
printf("Block %d:",i);scanf("%d",&b[i]);
printf("Enter the size of the files :-\n"); for(i=1;i<=nf;i++)
{
printf("File %d:",i);
scanf("%d",&f[i]);
}
for(i=1;i<=nf;i++)
{
for(j=1;j<=nb;j++)
{
if(bf[j]!=1)
```

```

{
temp=b[j]-f[i]; if(temp>=0)
if(lowest>temp)
{
ff[i]=j;
lowest=temp;
}
}
}
frag[i]=lowest; bf[ff[i]]=1;
lowest=10000;
}
printf("\nFile No\tFile Size \tBlock No\tBlock Size\tFragment");
for(i=1;i<=nf && ff[i]!=0;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
getch();
}

```

OUTPUT:

INPUT

Enter the number of blocks: 3

Enter the number of files: 2

Enter the size of the blocks:-

Block 1: 5

Block 2: 2

Block 3: 7

Enter the size of the files:-

File 1: 1

File2: 4

OUTPUT:

File No	File Size	Block No	Block Size	Fragment
1	1	2	2	1
2	4	1	5	1

RESULT:

Thus, the Best-Fit memory allocation strategy is implemented successfully.

DATE:

EXPT. NO.: 7c

IMPLEMENTATION OF WORST FIT MEMORY ALLOCATION

AIM:

To implement the Worst-Fit memory allocation strategy.

ALGORITHM:

Step 1: Start.

Step 2: Input memory blocks and process sizes.

Step 3: For each process, find the largest available block.

Step 4: Allocate that block.

Step 5: Reduce the block size after allocation.

Step 6: Repeat for all processes.

Step 7: Stop.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
#define max 25
void main() {
int frag[max],b[max],f[max],i,j,nb,nf,temp;
static int bf[max],ff[max];
clrscr();
printf("\n\tMemory Management Scheme - First Fit"); printf("\nEnter the
number of blocks:");
scanf("%d",&nb);
printf("Enter the number of files:"); scanf("%d",&nf);
printf("\nEnter the size of the blocks:-\n");
for(i=1;i<=nb;i++)
{printf("Block %d:",i);
scanf("%d",&b[i]);}
printf("Enter the size of the files :-\n");
for(i=1;i<=nf;i++)
{printf("File %d:",i);
scanf("%d",&f[i]);}
for(i=1;i<=nf;i++)
```

```

{
for(j=1;j<=nb;j++){
if(bf[j]!=1){
temp=b[j]-f[i];
if(temp>=0)
{ff[i]=j; break;}} }
frag[i]=temp;
bf[ff[i]]=1;
}
printf("\nFile_no:\tFile_size:\tBlock_no:\tBlock_size:\tFragement");
for(i=1;i<=nf;i++)
printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
getch();
}

```

OUTPUT:

INPUT

Enter the number of blocks: 3
Enter the number of files: 2
Enter the size of the blocks:-
Block 1: 5
Block 2: 2
Block 3: 7
Enter the size of the files:-
File 1: 1
File 2: 4

OUTPUT

File No	File Size	Block No	Block Size	Fragment
1	1	1	5	4
2	4	3	7	3

RESULT:

Thus, the Worst-Fit memory allocation strategy is implemented successfully.

DATE: / /

EXPT. NO: 8a

PAGE REPLACEMENT ALGORITHM – FIFO

AIM:

To implement page replacement algorithms.

ALGORITHM:

1. Start the program.
2. Get the number of pages.
3. Get the pages in numbered order.
4. Get the number of frames and assign each frame initially to 1.
5. Place the incoming pages in a queue.
6. Now in case of a new page, the page at the head of the queue is removed and the new page is inserted at the tail of the queue.
7. If the page is already in queue, page fault is incremented.
8. The process is repeated until the number of pages get exhausted.
9. Print the page fault.
10. Stop the program.

PROGRAM:

```
#include<stdio.h>

#include<conio.h>

void main()
{
int i,j,k,n,no,a[20],frame[20],avail,count=0;
clrscr();
```

```
printf("enter no. of pages:\n");
scanf("%d",&n);
printf("enter page no.:\n");
for(i=1;i<=n;i++)
scanf("%d",&a[i]);
printf("enter no. of frames:\n");
scanf("%d",&no);
for(i=0;i<no;i++)
frame[i]=-1;
j=0;
printf("ref string\tpage frames\n");
for(i=1;i<=n;i++) {
printf("%d\t\t",a[i]);
avail=0;
for(k=0;k<no;k++)
if(frame[k]==a[i])
avail=1;
if(avail==0) {
frame[j]=a[i];
j=(j+1)%no;
count++;
for(k=0;k<no;k++)
printf("%d\t",frame[k]);
}
printf("\n");
}
```



```
printf("page fault is %d",count);  
getch();  
}
```

OUTPUT:

Enter no. of pages : 10

Enter page no. : 7 0 1 2 0 3 0 4 2 3

enter no. of frames : 3

ref string	page frames		
7	7	-1	-1
0	7	0	-1
1	7	0	1
2	2	0	1
0			
3	2	3	1
0	2	3	0
4	4	3	0
2	4	2	0
3	4	2	3

page fault is 9

RESULT:

The FIFO (First-In First-Out) Page Replacement Algorithm was implemented successfully and the number of page faults was calculated for the given reference string and memory frames.

DATE:

EXPT. NO: 8b

PAGE REPLACEMENT ALGORITHM-LRU

AIM:

To implement page replacement algorithm LRU.

ALGORITHM:

1. Start the program.
2. Get the number of reference strings from the user.
3. Get the number of frames from the user.
4. Get the reference strings from the user.
5. Create a stack.
6. When the page fault occurs replace page present at the bottom of the stack.
7. If the page is already in stack, page fault is incremented.
8. The process is repeated until the number of pages get exhausted.
9. Print the page fault.
10. Stop the program.

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
void main(){
int n,i,j,f,q[20],p[20],pgf=0,st=0,t,cnt;
clrscr();
printf("enter no. of ref strings:\n");
scanf("%d",&n);
```

```
printf("enter no. of frames:\n");
scanf("%d",&f);
printf("enter strings:\n");
for(i=0;i<n;i++)
scanf("%d",&q[i]);
for(i=0;i<f;i++)
p[i]=999;
for(i=0;i<n;i++)
{
for(j=0;j<f;j++)
if(p[j]==q[i])
{
st++;
t=p[j];
cnt=j;
}
if(st==0)
{
printf("\n fault \n");
pgf++;
for(j=f;j>0;j--)
p[j]=p[j-1];
p[0]=q[i];
}
else
{
```

```

printf("\n");
for(j=cnt;j>0;j--)
p[j]=p[j-1];
p[0]=t;
st=0;

}
for(j=0;j<f;j++)
printf("%d\t",p[j]); }
printf("\n tot pag fault is %d",pgf); getch();
}

```

OUTPUT:

enter no. of ref strings: 10

enter no. of frames: 3

enter strings: 7 0 1 2 0 3 0 4 2 3

fault

7	999	999
---	-----	-----

Fault

0	7	999
---	---	-----

Fault

1	0	7
---	---	---

Fault

2	1	0
---	---	---

0	2	1
---	---	---

Fault

3 0 2

0 3 2

Fault

4 0 3

Fault

2 4 0

Fault

3 2 4

tot pag fault is 8

RESULT:

The LRU (Least Recently Used) Page Replacement Algorithm was implemented successfully and the number of page faults was calculated for the given reference string and memory frames.

DATE:

EXPT. NO: 8c

PAGE REPLACEMENT ALGORITHM-LFU

AIM:

To implement page replacement algorithm LFU.

ALGORITHM:

1. Start the program.
2. Enter the number of ages.
3. Enter the number of frames.
4. Assign each frame the value -1.
5. Declare a variable count to determine the frequency of the page and initially assign it to zero
6. Get the pages in numbered order.
7. Now enter the pages in a queue. If a new page arrives, then check the queue for the page
8. with the least value of count and then replace it.
9. If the page is already in queue, page fault is incremented.
10. Repeat the process till the pages get exhausted.
11. Print the page fault.
12. Stop the program.

PROGRAM:

```
#include<stdio.h>
```

```
int main(){
```

```
int f,p;
```

```
int pages[50],frame[10],hit=0,count[50],time[50];
```

```
int i,j,page,flag,least,minTime,temp;
```

```
printf("Enter no of frames : ");
scanf("%d",&f);
printf("Enter no of pages : ");
scanf("%d",&p);
for(i=0;i<f;i++){
frame[i]=-1;
}
for(i=0;i<50;i++){
count[i]=0;
}
printf("Enter page no : \n");
for(i=0;i<p;i++){
scanf("%d",&pages[i]);
}
printf("\n");
for(i=0;i<p;i++){
count[pages[i]]++;
time[pages[i]]=i;
flag=1;
least=frame[0];
for(j=0;j<f;j++){
if(frame[j]==-1 || frame[j]==pages[i]){
if(frame[j]!=-1){
hit++;
}
}
flag=0;
```

```

frame[j]=pages[i];
break;
}
if(count[least]>count[frame[j]]){
least=frame[j];
}
}
if(flag){
minTime=50;
for(j=0;j<f;j++){
if(count[frame[j]]==count[least] && time[frame[j]]<minTime){
temp=j;
minTime=time[frame[j]];
}
}
count[frame[temp]]=0;
frame[temp]=pages[i];
}
for(j=0;j<f;j++)
{
printf("%d ",frame[j]);
}
printf("\n");
}
printf("Page hit = %d",hit);
return 0;}

```


OUTPUT:

Enter no. of frames : 3

Enter no. of pages : 10

Enter page no.;

2 3 4 2 1 3 7 5 4 3

Frames:

2	1	-1
---	---	----

2	3	-1
---	---	----

2	3	4
---	---	---

2	3	4
---	---	---

2	1	4
---	---	---

2	1	3
---	---	---

2	7	3
---	---	---

2	7	5
---	---	---

2	4	5
---	---	---

2	4	3
---	---	---

Page hit = 1

RESULT:

The LFU (Least Frequently Used) Page Replacement Algorithm was implemented successfully and the number of page faults was calculated for the given reference string and memory frames.

DATE:

EXPT. NO: 9a

IMPLEMENTATION OF DISK SCHEDULING ALGORITHM- FCFS SCHEDULING ALGORITHM

AIM:

To implement FCFS (First Come First Serve) Disk Scheduling Algorithm and calculate the total head movement with detailed head movement output.

ALGORITHM:

1. Start the program.
2. Input number of disk requests.
3. Enter the request queue.
4. Input initial head position.
5. Initialize total head movement to 0.
6. Move the head sequentially to each request in arrival order.
7. Calculate movement using absolute difference.
8. Add movement to total head movement.
9. Display total head movement.
10. Stop the program.

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

int main() {

int queue[100], n, head, i, seek = 0, diff;

printf("Enter number of requests: ");

scanf("%d", &n);
```

```

printf("Enter request queue: ");
for(i = 0; i < n; i++) {
scanf("%d", &queue[i]);
}
printf("Enter initial head position: ");
scanf("%d", &head);
printf("\n--- Step-by-Step Head Movements ---\n");
for(i = 0; i < n; i++) {
diff = abs(queue[i] - head);
seek += diff;
printf("Head moves from %d to %d (Movement: %d)\n", head, queue[i], diff);
head = queue[i]; // Update head to current track
}
printf("\nTotal head movement: %d cylinders\n", seek);
return 0;
}

```

OUTPUT:

```

Enter number of requests: 8
Enter request queue: 98 183 37 122 14 124 65 67
Enter initial head position: 53
--- Step-by-Step Head Movements ---
Head moves from 53 to 98 (Movement: 45)
Head moves from 98 to 183 (Movement: 85)

```

Head moves from 183 to 37 (Movement: 146)

...

Total head movement: 640 cylinders

RESULT:

The FCFS Disk Scheduling Algorithm was implemented successfully and the total head movement was calculated by servicing disk requests in the order of arrival.

DATE:

EXPT. NO: 9b

IMPLEMENTATION OF DISK SCHEDULING ALGORITHM- SSTF SCHEDULING ALGORITHM

AIM:

To implement SSTF (Shortest Seek Time First) Disk Scheduling Algorithm and calculate the total head movement.

ALGORITHM:

1. Start the program.
2. Input number of disk requests.
3. Enter the request queue.
4. Input initial head position.
5. Initialize total head movement to 0.
6. Select the request closest to the current head position.
7. Move the head to that request.
8. Calculate head movement and add it to total movement.
9. Mark the serviced request.
10. Repeat until all requests are serviced.
11. Display total head movement.
12. Stop the program.

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

#include <limits.h>

int main() {
```

```

int n, head, total_movement = 0;
printf("Enter number of requests: ");
scanf("%d", &n);
int requests[n], visited[n];
printf("Enter request queue: ");
for(int i = 0; i < n; i++) {
scanf("%d", &requests[i]);
visited[i] = 0;
}
printf("Enter initial head position: ");
scanf("%d", &head);
printf("\n--- SSTF Seek Sequence ---\n");
for(int i = 0; i < n; i++) {
int min_dist = INT_MAX, index = -1;
for(int j = 0; j < n; j++) {
if(!visited[j]) {
int dist = abs(requests[j] - head);
if(dist < min_dist) {
min_dist = dist;
index = j;
}
}
}
visited[index] = 1;
total_movement += min_dist;
printf("Head moves from %d to %d (Movement: %d)\n", head,
requests[index], min_dist);

```

```
head = requests[index];  
}  
printf("\nTotal Seek Operations: %d cylinders\n", total_movement);  
return 0;  
}
```

OUTPUT:

Enter number of requests: 8

Enter request queue: 98 183 37 122 14 124 65 67

Enter initial head position: 53

--- SSTF Seek Sequence ---

Head moves from 53 to 65 (Movement: 12)

Head moves from 65 to 67 (Movement: 2)

Head moves from 67 to 37 (Movement: 30)

Head moves from 37 to 14 (Movement: 23)

Head moves from 14 to 98 (Movement: 84)

Head moves from 98 to 122 (Movement: 24)

Head moves from 122 to 124 (Movement: 2)

Head moves from 124 to 183 (Movement: 59)

Total Seek Operations: 236 cylinders.

RESULT:

The SSTF Disk Scheduling Algorithm was implemented successfully and the total head movement was calculated by servicing the nearest disk request first.

DATE: .

EXPT. NO: 9c

IMPLEMENTATION OF DISK SCHEDULING ALGORITHM- SCAN SCHEDULING ALGORITHM

AIM:

To implement the SCAN Disk Scheduling Algorithm and calculate the total head movement.

ALGORITHM:

1. Start the program.
2. Input number of disk requests.
3. Enter the request queue.
4. Input initial head position and direction of movement.
5. Sort the request queue.
6. Move the disk head in the given direction servicing all requests.
7. When the end is reached, reverse the direction.
8. Continue servicing remaining requests.
9. Calculate total head movement.
10. Display total head movement.
11. Stop the program.

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

void sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
            }
}

int main() {
    int n, head, disk_size, total_seek = 0, dloc;
    printf("Enter disk size (e.g., 200): ");
    scanf("%d", &disk_size);
    printf("Enter number of requests: ");
    scanf("%d", &n);
    int queue[n + 2];
    printf("Enter request queue: ");
    for(int i = 0; i < n; i++) scanf("%d", &queue[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    queue[n] = head;
    queue[n + 1] = disk_size - 1;
    n += 2;
    sort(queue, n);
    for(int i = 0; i < n; i++) {
```

```

if(head == queue[i]) { dloc = i; break; }
}
printf("\n--- SCAN Head Movements (Moving Right) ---\n");
for(int i = dloc + 1; i < n; i++) {
    int diff = abs(queue[i] - head);
    total_seek += diff;
    printf("Head moves from %d to %d (Movement: %d)\n", head, queue[i], diff);
    head = queue[i];
}
for(int i = dloc - 1; i >= 0; i--) {
    int diff = abs(queue[i] - head);
    total_seek += diff;
    printf("Head moves from %d to %d (Movement: %d)\n", head, queue[i], diff);
    head = queue[i];
}
printf("\nTotal head movement: %d cylinders\n", total_seek);
return 0;
}

```

OUTPUT:

Enter disk size: 200

Enter number of requests: 8

Enter request queue: 98 183 37 122 14 124 65 67

Enter initial head position: 53

--- SCAN Head Movements ---

Head moves from 53 to 65 (Movement: 12)

Head moves from 65 to 67 (Movement: 2)

...

Head moves from 183 to 199 (Movement: 16)

Head moves from 199 to 37 (Movement: 162)

Head moves from 37 to 14 (Movement: 23)

Total head movement: 331 cylinders.

RESULT:

The SCAN Disk Scheduling Algorithm was implemented successfully and the total head movement was calculated by moving the disk head in one direction and servicing requests along the path.

DATE:

EXPT. NO: 9d

IMPLEMENTATION OF DISK SCHEDULING ALGORITHM- LOOK SCHEDULING ALGORITHM

AIM:

To implement the LOOK Disk Scheduling Algorithm and calculate the total head movement.

ALGORITHM:

1. Start the program.
2. Input number of disk requests.
3. Enter the request queue.
4. Input initial head position and direction of movement.
5. Sort the request queue.
6. Move the disk head in the selected direction servicing requests.
7. Stop at the last request in that direction (do not go to disk end).
8. Reverse the direction and service remaining requests.
9. Calculate total head movement.
10. Display total head movement.
11. Stop the program.

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

void sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
```

```

if (arr[j] > arr[j + 1]) {
int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
}
}

int main() {
int n, head, total_seek = 0, dloc;
printf("Enter number of requests: ");
scanf("%d", &n);
int queue[n + 1];
printf("Enter request queue: ");
for(int i = 0; i < n; i++) scanf("%d", &queue[i]);
printf("Enter initial head position: ");
scanf("%d", &head);
queue[n] = head;
sort(queue, n + 1);
for(int i = 0; i <= n; i++) {
if(head == queue[i]) { dloc = i; break; }
}

printf("\n--- LOOK Head Movements (Moving Right) ---\n");
for(int i = dloc + 1; i <= n; i++) {
int diff = abs(queue[i] - head);
total_seek += diff;
printf("Head moves from %d to %d (Movement: %d)\n", head, queue[i], diff);
head = queue[i];
}

for(int i = dloc - 1; i >= 0; i--) {

```

```

int diff = abs(queue[i] - head);
total_seek += diff;
printf("Head moves from %d to %d (Movement: %d)\n", head, queue[i], diff);
head = queue[i];
}
printf("\nTotal head movement: %d cylinders\n", total_seek);
return 0;
}

```

OUTPUT:

Enter number of requests: 8

Enter request queue: 98 183 37 122 14 124 65 67

Enter initial head position: 53

--- LOOK Head Movements (Moving Right) ---

Head moves from 53 to 65 (Movement: 12)

Head moves from 65 to 67 (Movement: 2)

Head moves from 67 to 98 (Movement: 31)

Head moves from 98 to 122 (Movement: 24)

Head moves from 122 to 124 (Movement: 2)

Head moves from 124 to 183 (Movement: 59)

Head moves from 183 to 37 (Movement: 146)

Head moves from 37 to 14 (Movement: 23)

Total head movement: 299 cylinders

RESULT:

The LOOK Disk Scheduling Algorithm was implemented successfully and the total head movement was calculated by servicing requests up to the last track in each direction.

DATE: .

EXPT. NO: 10a

FILE ALLOCATION STRATEGIES - SEQUENTIAL

AIM:

To write a program a program to implement contiguous file allocation.

ALGORITHM:

1. Start the process
2. Declare the size
3. Get the number of pages to be inserted
4. Get the value
5. Declare counter and stack
6. Select the least recently used page by counter value
7. Stack them according the selection.
8. Display the values
9. Stop the process

PROGRAM:

```
#include<stdio.h>

#include<conio.h>

void main(){
int f[50],i,st,j,len,c,k;
clrscr();
```



```
for(i=0;i<50;i++)
f[i]=0;
X:
printf("\n Enter the starting block & length of file");
scanf("%d%d",&st,&len);
for(j=st;j<(st+len);j++)
if(f[j]==0){
f[j]=1;
printf("\n%d->%d",j,f[j]);
}else{
printf("Block already allocated");
break;
}
if(j==(st+len))
printf("\n the file is allocated to disk");
printf("\n if u want to enter more files?(y-1/n-0)");
scanf("%d",&c);
if(c==1)
goto X;
else
exit();
getch();
}
```

OUTPUT:

Enter the starting block and length of file: 4 10

4->1

5->1

6->1

7->1

8->1

9->1

10->1

11->1

12->1

13->1

The file is allocated to disk

If u want to enter more files?(y-1/n-0)

RESULT:

Thus the program was executed successfully and hence output verified.

DATE:

EXPT. NO: 10b

FILE ALLOCATION STRATEGIES – INDEXED

AIM:

To write a program a program to implement indexed file allocation

ALGORITHM:

1. Start the process
2. Declare the size
3. Get the number of pages to be inserted
4. Get the value
5. Declare counter and stack
6. Select the index file used page by counter value
7. Stack them according the selection.
8. Display the values
9. Stop the process

PROGRAM:

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int f[50],i,k,j,inde[50],n,c,count=0,p;
```

```
void main(){
```

```
clrscr();
```

```
for(i=0;i<50;i++)
```

```
f[i]=0;
```

```
x:
```

```

printf("enter index block\t");
scanf("%d",&p);
if(f[p]==0){
f[p]=1;
printf("enter no of files on index\t");
scanf("%d",&n);
} else{
printf("Block already allocated\n");
goto x;
}
for(i=0;i<n;i++)
scanf("%d",&inde[i]);
for(i=0;i<n;i++)
if(f[inde[i]]==1){
printf("Block already allocated");
goto x;
}
for(j=0;j<n;j++)
f[inde[j]]=1;
printf("\n allocated");
printf("\n file indexed");
for(k=0;k<n;k++)
printf("\n %d->%d:%d",p,inde[k],f[inde[k]]);
printf(" Enter 1 to enter more files and 0 to exit\t");
scanf("%d",&c);
if(c==1)

```

```
goto x;  
else  
exit(); getch();  
}
```

OUTPUT:

Enter index block: 9

Enter no. of files on index : 3

1 2 3

Allocated

File indexed

9->1:1

9->2:1

9->3:1

Enter 1 to enter more files and 0 to exit

RESULT:

The Indexed File Allocation Strategy was implemented successfully and files were allocated using an index block that stores the addresses of all file blocks for efficient direct access.

DATE:

EXPT. NO: 10c

FILE ALLOCATION STRATEGIES - LINKED

AIM:

To write a program a program to implement linked file allocation.

ALGORITHM:

1. Start the process
2. Declare the size
3. Get the number of pages to be inserted
4. Get the value
5. Declare counter and stack
6. Select the linked file used page by counter value
7. Stack them according the selection.
8. Display the values
9. Stop the process

PROGRAM:

```
#include<stdio.h>

#include<conio.h>

void main(){
int f[50],p,i,j,k,a,st,len,n,c;
clrscr();
for(i=0;i<50;i++)
f[i]=0;
printf("Enter how many blocks that are already allocated");
```

```

scanf("%d",&p);
printf("\nEnter the blocks no.s that are already allocated");
for(i=0;i<p;i++){
scanf("%d",&a);
f[a]=1;
}
X:
printf("Enter the starting index block & length");
scanf("%d%d",&st,&len);
k=len;
for(j=st;j<(k+st);j++){
if(f[j]==0){
f[j]=1;
printf("\n%d->%d",j,f[j]);
}else{
printf("\n %d->file is already allocated",j);
k++;
}}
printf("\n If u want to enter one more file? (yes-1/no-0)");
scanf("%d",&c);
if(c==1)
goto X;
else
exit();
getch( );
}

```

OUTPUT:

Enter how many blocks that are already allocated: 3

Enter the blocks no.s that are already allocated: 4 7 9

Enter the starting index block & length: 3 7

3->1

4->file is already allocated

5->1

6->1

7->file is already allocated

8->1

9->file is already allocated

10->1

11->1

12->1

If u want to enter one more file?(yes-1/no-0)

RESULT:

Thus the program was executed successfully and hence output verified.